PySCIPOpt Workshop

Part 1: Modeling Basics

SCIP Workshop

Objectives

By the end of Part 1 you will be able to:

- Formulate **Linear Programs (LP)**, **Integer Programs (IP)**, and **Mixed-Integer Programs (MIP)** in PySCIPOpt.
- Translate a mathematical model into Python code.
- Solve classical optimization problems:
  - Transportation, Blending, Set Cover,
  - Knapsack, Facility Location, Graph Coloring.
- Extract solutions and solver statistics.

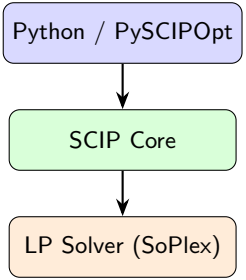
What is SCIP / PySCIPOpt?

SCIP — *Solving Constraint Integer Programs*

- One of the fastest non-commercial MIP solvers.
- Supports LP, MIP, MINLP, CIP.
- Open-source (Apache 2.0 license).
- Highly extensible plugin architecture.

PySCIPOpt — Python interface to SCIP

- Pythonic API for model building.
- Access to advanced features: callbacks, heuristics, separators, etc.
- Install: `pip install pyscipopt`



Model Lifecycle

Create Model → **Add Variables** → **Add Constraints** → **Optimize** → **Query Solution**

```
from pyscipopt import Model

model = Model("my_model")        # 1. Create
x = model.addVar(...)            # 2. Variables
model.addCons(...)               # 3. Constraints
model.optimize()                 # 4. Optimize
print(model.getVal(x))           # 5. Query
```

Variables

`model.addVar(name, vtype, lb, ub, obj)`

vtype	Domain	Typical use
"C"	Continuous ($\mathbb{R}$)	Quantities, flows
"B"	Binary ($\{0, 1\}$)	Yes/no decisions
"I"	Integer ($\mathbb{Z}$)	Counts

```
# Binary variable with objective coefficient 3
x = model.addVar(name="x", vtype="B", obj=3)

# Continuous variable in [0, 10]
y = model.addVar(name="y", vtype="C", lb=0, ub=10)

# Integer variable, default bounds [0, +inf)
z = model.addVar(name="z", vtype="I", obj=-2)
```

Constraints

`model.addCons(expression, name)`

Use `quicksum()` for efficient summation over index sets.

```
from pyscipopt import quicksum

# Single constraint
model.addCons(x + 2*y <= 10, name="capacity")

# Indexed family of constraints
for j in customers:
    model.addCons(
        quicksum(x[i,j] for i in suppliers) == demand[j],
        name=f"demand_{j}"
    )

# quicksum is preferred over Python's sum() for performance
model.addCons(quicksum(c[i]*x[i] for i in I) <= b)
```

Objective Function

Two ways to set the objective:

Option A: Via `obj` parameter in `addVar`

```
x = model.addVar(name="x", vtype="B", obj=3)  # coeff = 3
y = model.addVar(name="y", vtype="B", obj=2)  # coeff = 2
model.setMaximize()  # max 3x + 2y
```

Option B: Via `setObjective`

```
model.setObjective(
    quicksum(cost[i,j] * x[i,j] for i in I for j in J),
    sense="minimize"
)
```

Convenience methods:

- `model.setMinimize()` / `model.setMaximize()`

Exercise 1: First Model

Task

Build and print the following model (do **not** solve yet):

$$\begin{aligned} \max \quad & 3x + 2y \\ \text{s.t.} \quad & x + y \leq 1 \\ & 2x + y \leq 2 \\ & x, y \in \{0, 1\} \end{aligned}$$

- File: `ex01_first_model/first_model.py`
- Test: `python ex01_first_model/test_first_model.py`

Solving and Querying

```
model.hideOutput()          # suppress solver log (optional)
model.optimize()            # solve the model

status = model.getStatus()     # "optimal", "infeasible", ...
obj    = model.getObjVal()     # objective value
val_x  = model.getVal(x)      # value of variable x
```

Common status strings:

Status	Meaning
`"optimal"`	Optimal solution found
`"infeasible"`	No feasible solution exists
`"unbounded"`	Objective is unbounded
`"timelimit"`	Time limit reached (may have incumbent)

Exercise 2: Solving

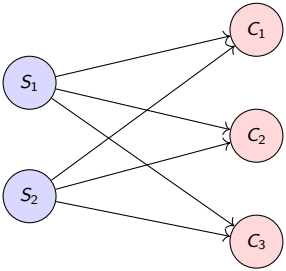
Task

Solve the model from Exercise 1 and extract:

- Optimal status, objective value, variable values.
- Solver statistics (number of nodes, LP iterations).

- File: `ex02_solving/solving.py`
- Test: `python ex02_solving/test_solving.py`

Transportation Problem



Sets: suppliers I, customers J
Parameters: supply s_i, demand d_j, cost c_{ij}
Variables: $x_{ij} \geq 0$ (flow from i to j)

$$
\begin{aligned}
\min \quad & \sum_{i \in I} \sum_{j \in J} c_{ij}\, x_{ij} & \\
\text{s.t.} \quad & \sum_{j \in J} x_{ij} \leq s_i & \forall\, i \in I \\
& \sum_{i \in I} x_{ij} \geq d_j & \forall\, j \in J \\
& x_{ij} \geq 0 &
\end{aligned}
$$

Data in Python — Dictionaries and Loops

```
# Data as dictionaries
supply   = {"S1": 100, "S2": 150}
demand   = {"C1": 80, "C2": 90, "C3": 70}
cost     = {("S1","C1"): 4, ("S1","C2"): 6, ("S1","C3"): 9,
            ("S2","C1"): 5, ("S2","C2"): 3, ("S2","C3"): 7}

# Variables via nested comprehension
x = {(i,j): model.addVar(name=f"x_{i}_{j}", vtype="C")
     for i in supply for j in demand}

# Constraints via loops
for i in supply:
    model.addCons(
        quicksum(x[i,j] for j in demand) <= supply[i])
for j in demand:
    model.addCons(
        quicksum(x[i,j] for i in supply) >= demand[j])
```

Exercise 3: Transportation

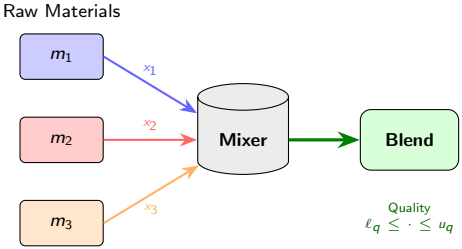
Task

Implement the transportation problem:

- Read data from provided dictionaries.
- Build variables, supply/demand constraints, minimize cost.
- Print optimal shipment plan.

- File: `ex03_transportation/transportation.py`
- Test: `python ex03_transportation/test_transportation.py`

Blending Problem



Combine raw materials to meet quality specs at minimum cost.

Sets: materials M, quality attributes Q

Parameters: cost c_m, content a_{mq}, bounds $[\ell_q, u_q]$, availability b_m

Variables: $x_m \geq 0$ (amount of material m)

$$
\begin{aligned}
\min \quad & \sum_{m \in M} c_m x_m \\
\text{s.t.} \quad & \ell_q \leq \frac{\sum_m a_{mq} x_m}{\sum_m x_m} \leq u_q && \forall q \in Q \\
& 0 \leq x_m \leq b_m && \forall m \in M
\end{aligned}
$$

Linearize: $\ell_q \sum_m x_m \leq \sum_m a_{mq} x_m \leq u_q \sum_m x_m$

Exercise 4: Blending

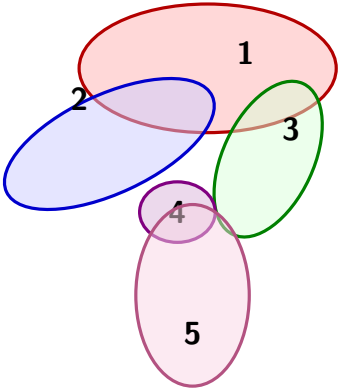
Task

Implement the blending problem:

- Model quality bounds as linear constraints (after linearization).
- Handle material availability limits.
- Minimize total cost.

- File: `ex04_blending/blending.py`
- Test: `python ex04_blending/test_blending.py`

From LP to IP: Set Cover



Select fewest sets to cover all elements.

Given: Universe $U = \{1, \ldots, n\}$, subsets $S_1, \ldots, S_m \subseteq U$, costs c_j.
Goal: Select minimum-cost subsets that cover every element.

Variables: $x_j \in \{0, 1\}$ (select subset j?)

$$\begin{aligned} \min \quad & \sum_{j=1}^{m} c_j x_j & \\ \text{s.t.} \quad & \sum_{j:\, i \in S_j} x_j \geq 1 & \forall\, i \in U \\ & x_j \in \{0, 1\} & \forall\, j \end{aligned}$$

- This is a **pure integer program** (all variables binary).
- LP relaxation gives a lower bound; SCIP uses branch-and-bound.

Exercise 5: Set Cover

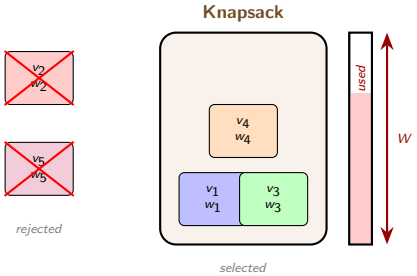
Task

Implement the set cover problem:

- Define binary variables for each subset.
- Add covering constraints for each element.
- Minimize total cost.

- File: `ex05_set_cover/set_cover.py`
- Test: `python ex05_set_cover/test_set_cover.py`

Knapsack Problem



Classic 0–1 Knapsack: select items to maximize value without exceeding capacity.

Applications: budgeting, cargo loading, resource allocation.

Variables: $x_j \in \{0, 1\}$ (take item j?)

$$
\begin{aligned}
\max \quad & \sum_{j=1}^{n} v_j \, x_j \\
\text{s.t.} \quad & \sum_{j=1}^{n} w_j \, x_j \leq W \\
& x_j \in \{0, 1\} \qquad \forall j
\end{aligned}
$$

where v_j = value, w_j = weight, W = capacity.

- NP-hard in general; SCIP handles it efficiently via specialized techniques.

Exercise 6: Knapsack

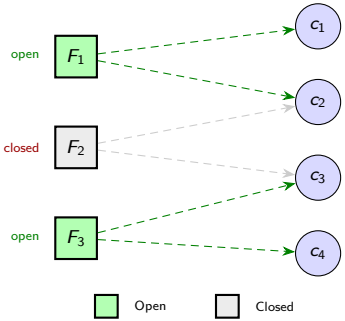
Task

Implement the 0–1 knapsack problem:

- Create binary variables for each item.
- Add the capacity constraint.
- Maximize total value and report selected items.

- File: `ex06_knapsack/knapsack.py`
- Test: `python ex06_knapsack/test_knapsack.py`

Facility Location



Decide which facilities to open and how to assign customers.

Sets: facilities F, customers C

Parameters: fixed cost f_i, assignment cost t_{ij}, demand d_j, capacity q_i

Variables: $y_i \in \{0,1\}$ (open i?), $x_{ij} \geq 0$ (fraction served)

$$
\begin{aligned}
\min \quad & \sum_{i \in F} f_i \, y_i + \sum_{i \in F} \sum_{j \in C} t_{ij} \, x_{ij} & \\
\text{s.t.} \quad & \sum_{i \in F} x_{ij} = 1 & \forall j \in C \\
& \sum_{j \in C} d_j \, x_{ij} \leq q_i \, y_i & \forall i \in F \\
& x_{ij} \geq 0, \; y_i \in \{0,1\} &
\end{aligned}
$$

- The constraint $x_{ij} \leq y_i$ **links** assignment to opening.

Exercise 7: Facility Location

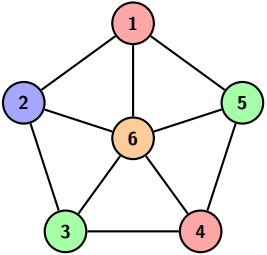
Task

Implement the uncapacitated facility location problem:

- Binary variables for facility opening decisions.
- Continuous variables for customer assignments.
- Linking and demand constraints.

- File: `ex07_facility_location/facility_location.py`
- Test: `python ex07_facility_location/test_facility_location.py`

Graph Coloring



4 colors — is this optimal?

Given: undirected graph $G = (V, E)$, set of colors $K = \{1, \ldots, k\}$.
Goal: minimize the number of colors used such that no two adjacent vertices share a color.

Variables: $x_{v,c} \in \{0, 1\}$ (vertex v gets color c?), $w_c \in \{0, 1\}$ (color c used?)

$$
\begin{aligned}
\min \quad & \sum_{c \in K} w_c \\
\text{s.t.} \quad & \sum_{c \in K} x_{v,c} = 1 && \forall\, v \in V \\
& x_{u,c} + x_{v,c} \leq w_c && \forall\, (u, v) \in E,\ c \in K \\
& x_{v,c},\ w_c \in \{0, 1\}
\end{aligned}
$$

Symmetry: permuting colors $\Rightarrow$ equivalent solution.
Break: $w_1 \geq w_2 \geq \cdots \geq w_k$.

Exercise 8: Graph Coloring

Task

Implement the graph coloring formulation:

- Assignment and conflict constraints.
- Minimize number of colors.
- Add symmetry-breaking constraints.

- File: `ex08_graph_coloring/graph_coloring.py`
- Test: `python ex08_graph_coloring/test_graph_coloring.py`

Part 1 — Wrap-up

What we covered:

- The PySCIPOpt modeling lifecycle: variables, constraints, objective.
- Solving and querying solutions and statistics.
- Classical LP/IP/MIP formulations:
  - Transportation (LP)
  - Blending (LP)
  - Set Cover (IP)
  - Knapsack (IP)
  - Facility Location (MIP)
  - Graph Coloring (IP)

Up next — Part 2: Advanced Topics

- Row generation (cutting planes, lazy constraints)
- Column generation
- Branch-and-Price